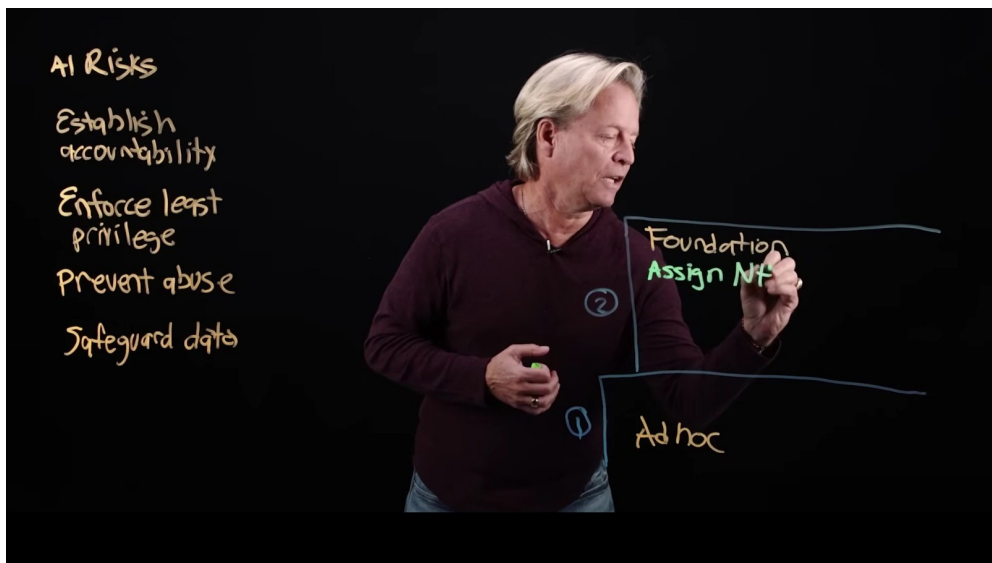


课程笔记

IBM 《AI 身份与访问管理：保护 Agent 系统的 4 个关键步骤》

AI 双语课 & Codex

2026 年 5 月 28 日



视频作者/频道: AI 双语课

发布日期: 2026 年 5 月 9 日

视频时长: 00:14:38

视频链接: <https://www.bilibili.com/video/BV1mco9B6Eib/>

字幕来源: 平台未提供可下载 CC 字幕; 本文基于英文 Whisper ASR、视频画面与人工整理生成。

目录

1 从成熟度模型看 Agentic IAM 的治理路线	2
1.1 问题入口：为什么 Agent 系统需要 IAM 成熟度路线	2
1.2 成熟度模型的来源：从 CMM 到安全治理阶梯	3
1.3 本章小结	3
2 风险地图：Agent 系统必须先回答的四个问题	4
2.1 问责：从项目级凭证转向 Agent 级责任	4
2.2 最小权限：避免超级 Agent 扩大攻击面	4
2.3 滥用防护：恶意与无意风险都要纳入设计	5
2.4 数据保护：同时校验用户与 Agent 的访问资格	5
2.5 本章小结	6
3 阶段一：Ad Hoc，风险意识尚未进入工程流程	7
3.1 最低成熟度：有 Agent，缺少控制面	7
3.2 业务压力如何挤压安全设计	8
3.3 本章小结	8
4 阶段二：Foundation，建立可识别、可委托、可审计的 Agent 访问	9
4.1 非人类身份：先让 Agent 在系统中有名字	9
4.2 委托与代表执行：把用户意图传递给 Agent 流程	10
4.3 SIEM 与审计：让访问链路留下证据	11
4.4 本章小结	12
5 阶段三：Enhanced，把 Agent 当作一等主体并收紧授权	12
5.1 一等主体：Agent 不再只是附属凭证	12
5.2 临时凭证：只为当前任务发放能力	13
5.3 精细化与上下文访问：按任务、时间与目标收缩权限	14
5.4 实时检测：从事后日志走向运行中发现异常	15
5.5 本章小结	16
6 阶段四：Adaptive，让认证、风险与撤销随流程动态变化	17
6.1 动态环境：非确定性流程要求持续判断	17
6.2 持续认证：每一步都重新确认 Agent 身份	18
6.3 基于风险的再认证：敏感操作提高验证强度	18
6.4 实时撤销：检测到异常就切断能力	19
6.5 本章小结	19
7 闭环：四个成熟度阶段如何回扣四类风险	20
7.1 从控制回到问责、最小权限、滥用防护与数据保护	20
7.2 渐进路线：先打基础，再持续增强	20
7.3 本章小结	21
8 总结与延伸	21

1 从成熟度模型看 Agentic IAM 的治理路线

1.1 问题入口：为什么 Agent 系统需要 IAM 成熟度路线

视频开场直接把问题放在企业安全治理的核心位置：当 AI Agent 开始调用工具、连接系统、读取数据、触发流程时，传统“应用接入系统”的权限管理思路会变得粗糙。身份与访问管理 (Identity and Access Management, IAM) 在这里指一整套治理能力：系统需要知道“谁或什么”正在行动，允许它在什么条件下访问哪些资源，并且能把这些决定留下可审计证据。对 Agentic 系统而言，“谁或什么”不再只包含人类用户和传统应用，还包含能够代表目标执行任务的软件行动者。

核心定义：Agentic IAM

IAM 是决定主体身份、访问边界、执行条件和审计证据的治理体系。Agent 是能够围绕某个目标调用工具、系统或数据并具有一定自主性的 AI 软件行动者。Agentic 系统则由一个或多个 Agent、工具、数据资源、工作流和委托关系组成；它的运行路径会随着上下文、模型输出和工具反馈发生变化。Agentic IAM 的任务，是把这些软件行动者纳入可识别、可授权、可追踪、可撤销的治理框架。

这个定义对后续章节很重要。传统 IAM 习惯围绕人、应用和服务账号建模；Agentic IAM 需要把 Agent 也视为治理对象。可以把传统应用想成一条固定生产线：输入、流程、输出大体稳定，权限边界可以较长期地绑定在应用或项目上。Agent 更像一个会临场选择路线的办事员：同一个目标可能触发不同工具调用、不同数据查询、不同下游流程。因此，治理重点从“这个项目能不能连上某个系统”推进到“这个 Agent 在这次任务、这个上下文、这段时间内能做什么”。

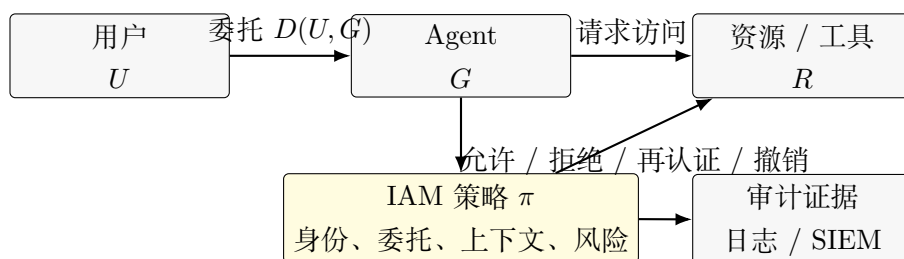


图 1: Agentic IAM 的基本治理对象：用户、Agent、资源和策略必须在同一个授权语境里被建模；图中的符号和执行动作会在后文逐级定义。

视频提出的路线是一套“四步成熟度系统”，用于让组织逐步管理 Agentic IAM，并让 AI 系统更能承受未来变化。这里的“逐步”并非降低安全要求；它承认工程组织通常无法一次性完成身份建模、授权收缩、实时检测、动态撤销等全部能力。成熟路线的价值在于给改造排序：先把主体识别出来，再让委托和审计可见，然后收紧凭证和权限，最后让认证、风险和撤销随着流程动态变化。

容易误判的人口问题

把 Agent 当作普通 API 调用方，会让权限设计停留在连接层：只要密钥能用，系统就放行。真正的风险在执行层：Agent 为什么调用、代表谁调用、当前任务是否需要这项权限、调用后是否能追责。后续四类风险都围绕这个执行层展开。

1.2 成熟度模型的来源：从 CMM 到安全治理阶梯

演讲者用能力成熟度模型（Capability Maturity Model, CMM）的历史来解释为什么这套 IAM 治理要被写成阶梯。CMM 的脉络来自 1986 年左右的能力成熟度研究，字幕还特别提到，这一早期研究由美国国防部资助。这个来源细节说明，成熟度模型从一开始就服务于复杂工程能力如何分级、评估和改进的问题；后续在 1991 年前后，它形成了影响广泛的成熟度模型范式。它关心的问题很朴素：一个组织能否从几乎没有体系的状态，逐步走向可重复、可度量、可改进的高级能力。

背景知识：成熟度模型

成熟度模型是一种分阶段改进框架，用来描述控制能力如何从临时实践成长为高级治理。它的相关概念是“能力等级”：每一级都增加一组更稳定的机制。它的反向思路是“一次性清单”：把所有控制项摊开后要求同时完成，常见结果是纸面合规很完整，运行时治理仍然松散。

把这个思路放到 Agentic IAM 中，四步路线就不只是目录结构。它表达的是安全控制的依赖关系：没有 Agent 身份，后续日志很难追到具体行动者；没有委托关系，系统无法判断 Agent 是否真的能代表用户访问数据；没有短生命周期凭证和细粒度授权，最小权限会停留在口号；没有实时检测和撤销，异常行为只能在事后复盘中被发现。成熟度模型把这些依赖关系排成可以落地的治理路径。

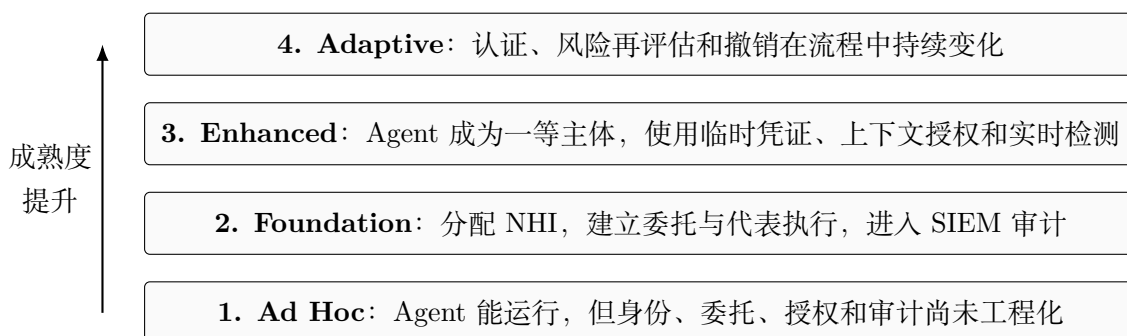


图 2: 本文使用的四级 Agentic IAM 成熟度阶梯。每一级都为下一层控制提供前置条件。

为了便于后续章节复用，可以把本视频的治理阶梯压缩成四级：Ad Hoc 表示 Agent 已经被构建出来，但身份、授权、审计和风险控制尚未系统化；Foundation 建立最小可用的可识别、可委托、可审计能力；Enhanced 把 Agent 当成一等主体，并通过临时凭证、上下文授权和实时检测收紧权限；Adaptive 让认证、风险评估和访问撤销在流程中持续变化。后续章节会逐级展开。

1.3 本章小结

本节给后续讨论定了三个坐标。第一，IAM 在 Agentic 场景中管理的不只是人类用户，也包括能调用工具和数据的软件行动者。第二，Agentic 系统的关键特征是多主体、多工具、多委托关系，以及运行路径的变化性。第三，成熟度模型提供治理排序：组织先建立基础识别和审计，再逐步走向细粒度授权、实时检测和动态响应。接下来需要回答的问题是：这条阶梯到底要降低哪些风险。

2 风险地图：Agent 系统必须先回答的四个问题

2.1 问责：从项目级凭证转向 Agent 级责任

成熟度路线要解决的第一类风险是问责。问责指系统能够判断一次行动应归属于哪个 Agent、哪个用户、哪个团队或哪个系统，并能给出足够证据。传统应用里，团队常用 API 密钥、功能账号或系统账号把系统连接起来：一个项目拿到一组凭证，应用用这组凭证访问数据库、消息队列、内部服务或第三方 API。对固定应用而言，这种做法常常能维持基本运转，因为责任边界大致落在项目团队。

核心定义：API 密钥、功能账号与问责

API 密钥是应用连接服务时使用的凭证，它证明“持有者可以访问”，但它本身不等同于行动者身份。功能账号或系统账号是给应用、进程、自动化任务使用的非个人账号，适合承载机器到机器的连接。问责是把一次访问、一次工具调用或一次流程启动，追溯到具体 Agent、代表的用户、所属团队和触发上下文的能力。

Agentic 系统会放大项目级凭证的模糊性。若多个 Agent 共用同一组项目密钥，日志里只能看到“项目 X 调用了系统 Y”，很难回答“哪个 Agent 为了哪个任务做了这次调用”。当系统发生错误查询、越权读取或错误流程启动时，项目级责任粒度过粗，调查会被迫依赖补充日志、推测和人工访谈。

这里还要定义一个底层概念：非确定性系统。非确定性系统是指在相似目标或相似输入下，运行路径、工具选择或输出结果可能发生变化的系统。确定性工作流像排好站点的列车，下一站大体已知；Agentic 流程更接近动态导航，系统可能根据途中反馈重新选择路线。IAM 设计必须假设“下一步不一定在设计时完全固定”，因此问责不能只绑定在项目或应用壳上。

问责风险的真实含义

项目团队仍然承担治理责任，但运行时证据需要下钻到 Agent 级别。只知道“哪个项目持有密钥”无法充分说明“哪个 Agent 在什么上下文中代表谁做了什么”。这就是视频把责任中心从项目级连接推进到 Agent 与 AI 控制面的原因。

2.2 最小权限：避免超级 Agent 扩大攻击面

第二类风险是最小权限。最小权限指只授予完成当前任务所需的最小访问能力。对 Agent 来说，这个原则比传统应用更紧，因为 Agent 往往能串联工具、读取多类数据、触发多个系统动作。一旦权限过宽，错误推理、提示注入、恶意指令或开发者无意设计缺陷都可能借助这组权限扩大影响范围。

演讲者用“超级 Agent”来描述这种危险状态。超级 Agent 是拥有过宽、多系统、边界不清权限的 Agent；“超级”听起来像能力强，安全上意味着爆炸半径大。一个只负责查询订单状态的 Agent，不应同时拥有修改付款信息、导出全部客户数据、创建管理员账号的权限。一个 Agent 应该聚焦单一任务，并且只拿到该任务必需的访问能力。

可以用一个很轻的形式化表达来帮助记忆。理想情况下，Agent G 在上下文 C 中执行任务 $task$ 时，拿到的权限集合应接近该任务的最小权限集合：

$$A(G, task, C) \approx A_{\min}(task, C)$$

其中：

- G ：执行任务的 Agent。
- $task$ ：当前要完成的具体任务。
- C ：上下文，包括代表的用户、时间、资源敏感度、工具链路和运行状态。
- $A(G, task, C)$ ：系统实际授予该 Agent 的权限集合。
- $A_{min}(task, C)$ ：完成该任务所需的最小权限集合。

核心定义：最小权限与超级 Agent

最小权限是把授权压缩到“当前任务刚好需要”的范围内。超级 Agent 是具有宽泛、跨系统或缺少边界的过度代理能力的 Agent。最小权限降低的是影响半径：即使 Agent 被误用、被诱导或被攻陷，攻击者能利用的权限也被限制在较小范围。

2.3 滥用防护：恶意与无意风险都要纳入设计

第三类风险是滥用防护。滥用防护指通过身份、授权、检测和阻断机制，限制 Agent 做出有害行为。视频特别强调两种来源：一种是恶意滥用，例如有人故意构建一个 Agent 去执行组织不允许的动作；另一种是无意滥用，例如开发者以为自己做了一个很好的 AI 用例，却让 Agent 连接到敏感系统，意外造成越权访问、数据泄露或流程误触发。

这一区分很实际。企业风险并不只来自“坏人写坏 Agent”。很多事故来自“好意图削弱边界”：开发者为了让体验顺畅，给 Agent 多开几个系统权限；为了降低失败率，让 Agent 在出错时自动重试和改写请求；为了提高自动化程度，让 Agent 直接启动下游流程。每个选择单看都有业务理由，组合起来会形成绕过审批、扩大访问、隐藏责任的路径。

相关概念：滥用、误用与防护栏

滥用偏向结果：Agent 的能力被用于有害或越权目的。误用偏向过程：设计者或使用者没有恶意，却把 Agent 放进了超出安全假设的场景。防护栏是约束 Agent 访问和行动的策略与执行机制，包括授权检查、上下文限制、异常检测、速率限制、人工审批和撤销能力。防护栏需要落到可执行控制上，不能只作为原则性口号。

后续成熟度路线会把滥用防护拆成多个层次：Foundation 阶段先让日志和审计可用；Enhanced 阶段加入实时检测，观察流程是否偏离预期；Adaptive 阶段在检测到风险时重新认证或撤销访问。这里先建立风险地图即可：无论动机恶意还是无意，Agent 的工具能力都需要被边界、证据和响应机制包住。

2.4 数据保护：同时校验用户与 Agent 的访问资格

第四类风险是数据保护。数据保护在本视频中有一个关键细节：当 Agent 代表用户查询数据、带回信息或启动流程时，系统需要同时确认用户有资格访问该数据，Agent 也有资格访问该数据。只检查用户会漏掉 Agent 自身权限边界；只检查 Agent 会漏掉“代表谁”的业务资格。

举一个企业内部例子：用户 U 让 Agent G 查询客户记录 R 。即便 U 是销售经理，也不代表任何 Agent 都可以读取完整客户档案；即便 G 被批准访问客户系统，也不代表它可以替所有用户读

取所有客户数据。正确的授权判断要同时看主体、资源和上下文：谁发起、哪个 Agent 执行、是否存在代表关系、数据敏感度如何、当前任务是否真的需要这些字段。

核心定义：数据保护

数据保护是确保数据检索、信息返回和流程启动都受到授权约束。Agent 代表用户行动时，访问决策至少要同时校验用户资格与 Agent 资格，并保留二者之间的代表关系。它的目标是阻止未经授权的数据访问，以及阻止 Agent 借合法入口拿到超出任务范围的数据。

从机制上看，数据保护与前面三类风险互相咬合。没有问责，系统不知道哪条 Agent 链路取走了数据；没有最小权限，Agent 可能拿到过多表、字段或 API 动作；没有滥用防护，异常查询模式无法及时触发阻断。因此，数据保护不能被压缩成单独的数据分类问题；它依赖身份、委托、授权、审计和响应机制共同工作。

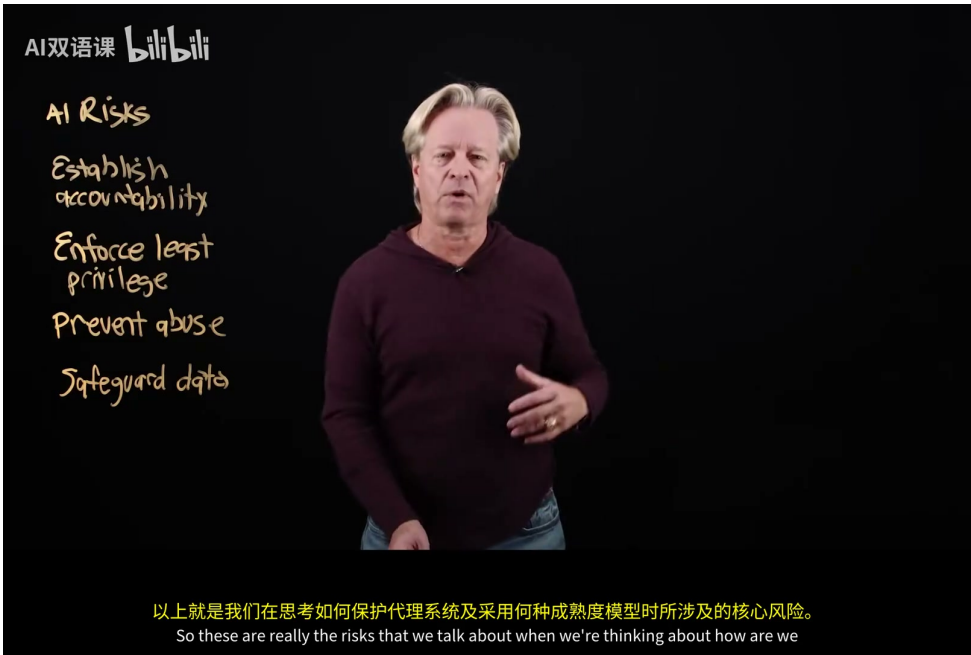


图 3: 视频中完整写出的四类 AI 风险：问责、最小权限、滥用防护与数据保护。¹

2.5 本章小结

本节把后续成熟度模型要压住的风险分成四类。问责要求系统能从项目级凭证下钻到 Agent 级行动证据；最小权限要求 Agent 只拥有当前任务需要的能力，避免形成超级 Agent；滥用防护同时覆盖恶意设计和无意误用；数据保护要求用户和 Agent 的访问资格被同时校验。四类风险共同说明一件事：Agentic IAM 的治理对象已经从“连接是否可用”推进到“行动是否可解释、可限制、可审计、可阻断”。下一阶段的 Ad Hoc 会展示最低成熟度状态下这些风险如何暴露出来。

¹ 视频画面时间区间：00:04:17–00:04:26。

3 阶段一：Ad Hoc，风险意识尚未进入工程流程

3.1 最低成熟度：有 Agent，缺少控制面

在前一章的风险地图里，Agentic 系统已经暴露出四个必须治理的问题：问责、最小权限、滥用防护与数据保护。视频在 00:04:28 之后把这些风险接到成熟度阶梯的最底层，称为 **Ad Hoc**。这个词在这里不应理解成“完全没有系统”。更准确的说法是：团队已经开始构建 Agent，系统也可能已经能调用工具、查询数据、发起流程，但身份、委托、授权、日志与审计这些控制点还没有进入工程设计。

换一个直观类比，Ad Hoc 像一条已经通车的临时道路：车可以走，货可以运，业务看起来跑起来了；可红绿灯、限速牌、车道边界、事故记录和执法证据还没有系统化部署。一旦发生越权访问、错误调用或敏感数据泄露，组织很难回答“哪个 Agent 做了什么、代表谁做、凭什么能做、事后靠什么复盘”。

Ad Hoc 的定义

Ad Hoc 是 Agentic IAM 的最低成熟度状态：Agent 正在被构建和运行，但非人类身份、委托链、授权边界、日志证据和风险控制尚未形成有意识的工程流程。它的核心风险在于“可运行”领先于“可治理”。



图 4: Ad Hoc 位于成熟度阶梯底部，下一步才进入 Foundation 控制层。²

视频里的表述很直接：团队“正在构建 Agent”，并且“有 AI 相关的压力”，于是重点放在让系统以非常基础的方式跑起来。这里的缺口不在模型能力本身，而在运行上下文。Agent 到底依赖哪个凭证访问系统？这个凭证属于项目、服务账号、某个用户，还是 Agent 自身？Agent 的一次任务是否被限制在单个目标内？系统是否知道它访问数据时代表谁？这些问题一旦没有显式答案，前一章的四类风险就会同时被放大。

² 视频画面时间区间：00:05:14–00:05:23。

Ad Hoc 最容易被误判的地方

看到 Agent 已经能完成任务，团队容易把“功能上线”误当成“安全就绪”。对 IAM 来说，真正的成熟度看的是访问链路能否被识别、限制、追踪和复盘。只要这些链路还靠默认凭证、共享账号、零散日志和人工记忆维持，系统仍处在 Ad Hoc。

3.2 业务压力如何挤压安全设计

Ad Hoc 的形成通常有一个很现实的原因：业务压力先到，安全设计后到。视频中说团队面对 AI 压力，想尽快做出系统、做出用例、证明 Agent 能产生价值。这种压力会把工程注意力挤到“让调用成功”上，例如把工具接起来、把 API key 放进去、把数据库查通、把流程跑完。短期看，这是产品探索所需的速度；长期看，它会把访问控制变成一堆事后补丁。

这类压力特别危险，因为 Agentic 系统的运行路径具有不确定性。传统脚本通常按固定步骤执行，权限边界还能靠流程图勉强推断。Agent 可能根据用户提示、工具返回、模型推理结果选择不同路径，今天只查一个知识库，明天可能调用工单系统、客户数据、邮件接口和自动化流程。控制面没有提前设计时，系统就会把“能连接到哪里”误当成“应该被允许做什么”。

控制面与执行面的区分

执行面关心任务是否完成，例如 Agent 是否查到数据、是否调用了工具、是否触发了流程。控制面关心任务以什么身份完成、权限从哪里来、访问是否符合策略、日志能否支撑问责。Ad Hoc 阶段常见的问题是执行面增长很快，控制面仍停留在项目级 API key、功能账号或临时日志。

因此，Ad Hoc 的教学价值在于暴露缺口。它是成熟度模型的起点，提示组织先承认已经有 Agent 在跑，再承认这些 Agent 的身份、权限和证据链还没有被严肃管理。这个承认很关键，因为下一阶段 Foundation 的所有动作都在回答同一个问题：怎样用最小一组基础控制，把 Agent 从“能跑”推进到“可识别、可委托、可审计”。

3.3 本章小结

Ad Hoc 阶段说明了 Agentic IAM 的底线问题：组织可能已经在构建 Agent，也可能已经让它们接入数据与流程，但风险意识尚未进入工程流程。它的典型特征包括共享或项目级凭证、模糊的执行上下文、缺少明确委托关系、授权边界粗糙、日志与审计证据不足。

这一阶段的价值在于给后续治理定锚。前一章提出的四类风险在这里都能找到具体落点：问责缺口来自身份不清，最小权限缺口来自权限过宽，滥用风险来自缺少检测与限制，数据保护风险来自用户与 Agent 的访问资格没有被同时校验。Foundation 阶段正是从这些缺口出发，先补上最基础的身份、委托和审计能力。

4 阶段二：Foundation，建立可识别、可委托、可审计的 Agent 访问

4.1 非人类身份：先让 Agent 在系统中有名字

视频在 00:05:22 之后进入第二级成熟度：**Foundation**。讲者把它定义为让系统“至少按我们想要的方式运行”的最少控制集合。这里的关键词是“最少”与“控制”。Foundation 不追求一步到位的高级自适应治理，它先把 Agentic 访问从黑箱变成可识别的工程对象。

第一个基础动作是给 Agent 分配 **非人类身份**，也就是 **Non-Human Identity, NHI**。NHI 指分配给软件行为体的身份，例如 Agent、工作负载、服务账号、自动化进程或后台任务。它和人类账号的共同点是都能作为访问控制的主体；差异在于生命周期、凭证形态和行为模式通常由系统生成和调度，而非由一个自然人手工登录。

非人类身份 (NHI)

NHI 是系统为软件行为体建立的可治理身份。对 Agent 来说，NHI 的作用是让系统能够表达“哪个 Agent 正在执行哪个动作、承担哪个任务、使用哪个权限”。没有 NHI，Agent 的行为会被折叠进项目、应用或共享凭证里，问责粒度会变粗。

这里要小心一个常见混淆：API key、功能账号、系统账号都可能让软件完成访问，但它们不天然等于成熟的 NHI 治理。凭证只是打开门的钥匙，身份是系统记录“谁在门口、为什么进门、进去后做了什么”的对象。Foundation 阶段至少要做到让 Agent 在 IAM 语境里有名字、有任务、有可记录的行为边界。

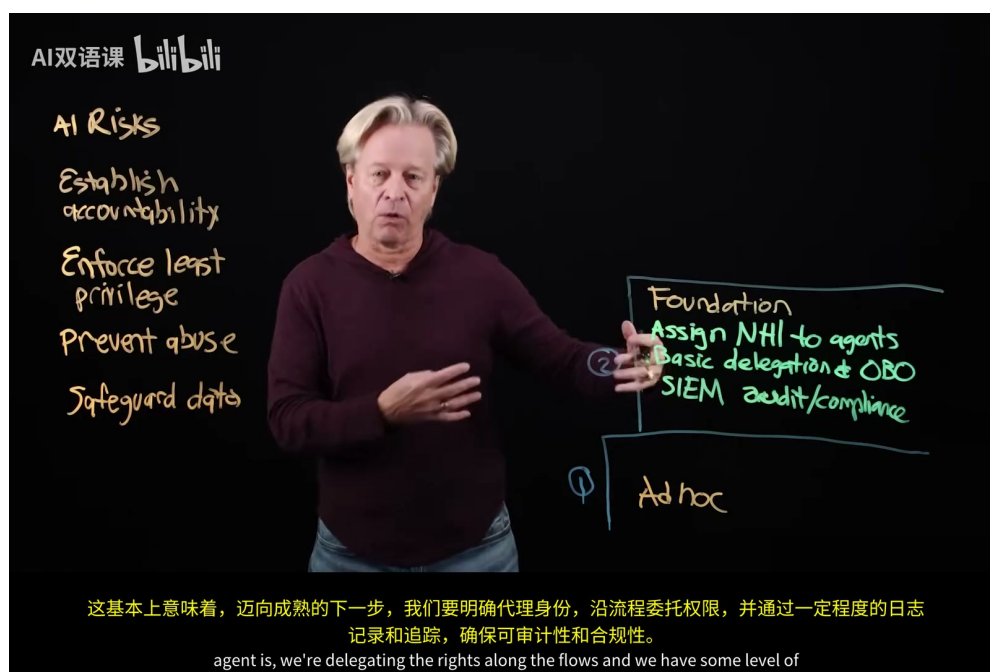


图 5: Foundation 阶段的三项基础控制：为 Agent 分配 NHI、建立委托与代表执行、把事件送入 SIEM 做审计与合规。³

³视频画面时间区间：00:07:08–00:07:18。

从前一章的风险地图看，NHI 首先修补的是问责。传统应用里，团队可能用项目级 API key 或功能账号连接系统；Agentic 系统里，Agent 的路径和输出可能每次变化，继续用粗粒度账号会让事件复盘变得含糊。给 Agent 建立 NHI，相当于给访问链路挂上第一块身份铭牌。

4.2 委托与代表执行：把用户意图传递给 Agent 流程

Foundation 的第二个动作是建立基本的 **委托与代表执行** 语义。视频中的例子很具体：用户在 Agentic 系统最前端发起提示，希望完成某件事；系统需要把这件事委托给 Agent，让 Agent 代表该用户工作。这里的“代表执行”，对应英文 *on behalf of*，意思是 Agent 的动作不能只记录成“Agent 做了某事”，还要保留它为谁而做、从谁那里获得了有限权限。

委托与代表执行

委托是把某个主体的有限权限交给另一个主体，用于完成有边界的任务。代表执行是委托链上的运行语义：Agent 可以执行动作，但审计和授权仍要保留上游主体，例如用户或上游 Agent。成熟系统应记录“谁委托、委托给谁、委托做什么、委托权限到哪里为止”。

可以用一个轻量公式压缩这个关系。系统在授权时需要看到的，不只是单个 Agent 身份，还包括用户到 Agent 的委托链以及当前上下文：

$$D(U, G) \Rightarrow P = (U \rightarrow G), \quad \text{allow}(P, A, R, C)$$

- U ：发起任务或受益于任务的人类用户。
- G ：执行任务的 Agent。
- $D(U, G)$ ：用户 U 向 Agent G 发出的有边界委托关系。
- $P = (U \rightarrow G)$ ：授权决策中的有效主体，表示 Agent 正在代表用户行动。
- A ：被请求的权限集合，例如读取、写入、发起流程或调用工具。
- R ：目标资源，例如数据、API、服务或业务流程。
- C ：上下文，例如任务目标、时间、数据敏感度、会话状态和委托链。
- $\text{allow}(P, A, R, C)$ ：在给定主体、权限、资源和上下文中允许访问的判定。

这个表达式不试图规定某个具体协议，它只说明 Foundation 阶段的最低语义要求：授权决策不能把用户意图、Agent 身份和上下文拆散。Agent 代表用户访问数据时，系统既要检查用户是否有资格访问，也要检查 Agent 是否被允许执行该类任务。前一章的数据保护风险正落在这里：只有用户有权限还不够，只有 Agent 有工具权限也不够，两者必须同时成立。

视频还补充了另一种场景：**自主系统**。自主系统指 Agent 不需要每一步都由用户直接提示，可能由系统计划、事件触发或其他 Agent 调度继续行动。即便在这种场景中，委托仍然存在，只是链条可能从用户到 Agent，变成 Agent 到 Agent，例如 $D(G_1, G_2)$ 。这时系统依然要维护“权利从哪里来、传给了谁、能做哪些动作”的链路。

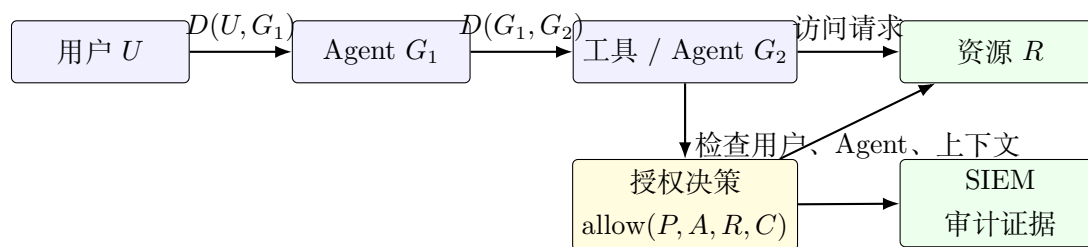


图 6: Foundation 阶段的委托链：下游授权要同时看到代表关系、Agent 身份、权限集合、资源和上下文。

代表执行不能被压扁成一个共享身份

如果所有下游访问都只显示为某个通用 Agent 账号，系统会丢失最重要的治理信息：上游用户是谁、委托是否仍有效、Agent 是否越过任务边界。这样的实现会让审计看似有日志，实际缺少可问责的上下文。

4.3 SIEM 与审计：让访问链路留下证据

Foundation 的第三个动作是把相关事件送入 **SIEM**。SIEM 的标准展开是 *Security Information and Event Management*，中文通常译为“安全信息与事件管理”。字幕里出现了“secure information and event management”的说法，这里采用安全行业更通用的标准术语。SIEM 的职责是收集、关联和分析安全日志与事件，让组织能从大量运行记录中看出访问链路、异常行为和合规证据。

SIEM、可审计性与合规

SIEM 是日志与安全事件的汇聚和分析系统。可审计性强调行动能够被追踪、复核和举证；它比普通可观测性更关注责任归属与证据链。合规指系统满足内部政策、监管要求或合同义务；在本视频语境里，合规依赖身份流、委托流和访问事件能够被记录并复盘。

Foundation 级别的审计不要求系统立即具备实时异常检测，那是后续 Enhanced 阶段会加强的能力。这里的底线是“有证据”。讲者在 00:07:06 之后总结说，这一级成熟度意味着系统知道 Agent 是谁，能够沿流程委托权利，并且拥有某种日志与追踪能力，从而支撑可审计性和合规。

具体到 Agentic IAM，进入 SIEM 的事件至少应该能回答几类问题：哪个 NHI 发起了动作？它代表哪个用户或上游 Agent？请求访问了哪个资源？使用了哪组权限？上下文是什么？访问被允许、拒绝还是出现异常？这些问题听起来像日志字段，实际是组织事后追责、发现滥用、证明合规的基本证据骨架。

Foundation 的三件事

Foundation 阶段建立 Agent 访问的最低治理面：第一，给 Agent 分配 NHI，使其可识别；第二，维护委托与代表执行语义，使权限来源可解释；第三，把访问事件送入 SIEM，使行为可审计并能服务合规。

有了这三件事，系统才从 Ad Hoc 的“临时可用”进入“基础可控”。它仍然可能权限粗、凭证寿命长、检测滞后，但它已经具备后续增强的地基：身份能被命名，权利能沿流程传递，行为能留下证据。

4.4 本章小结

Foundation 阶段是 Agentic IAM 的第一层工程地基。它不解决所有高级问题，却解决最先要落地的三个问题：让 Agent 在系统里有身份，让用户或 Agent 的权限能够被有边界地委托，让访问链路进入 SIEM 并形成审计证据。

本章定义了后续章节会继续使用的几个基础概念：非人类身份（NHI）让软件行为体成为可治理对象；委托描述权限如何从一个主体传给另一个主体；代表执行保留“为谁行动”的上游语义；自主系统说明无直接用户提示时仍要维护 Agent 间的授权链；SIEM、可审计性和合规规则把运行事件转化为组织可检查的证据。至此，Ad Hoc 暴露出的缺口被接上了基础控制面，下一阶段才能继续讨论更强的 Agent 主体治理、临时凭证、上下文访问和实时检测。

5 阶段三：Enhanced，把 Agent 当作一等主体并收紧授权

Foundation 阶段已经让 Agent 在系统里留下了名字、委托关系和审计证据。Enhanced 阶段继续向前走一步：企业不能只在日志里看见 Agent，还要在身份治理、凭证发放、授权决策和运行时检测中直接管理 Agent。换句话说，Agent 开始进入 IAM 的核心控制面，像人类账号、服务账号、工作负载身份一样成为可建模、可分配、可审查、可限制的主体。

这一层的关键难点来自 Agentic 系统的非确定性。同一个目标在两次运行中可能选择不同工具、访问不同资源、触发不同下游流程。长期有效、范围宽泛的凭证会把这种不确定性变成权限扩散；Enhanced 阶段的任务，就是把这种扩散压回到当前任务、当前上下文和当前资源边界之内。

5.1 一等主体：Agent 不再只是附属凭证

在 Foundation 阶段，非人类身份（NHI）先解决“系统能否分辨这个 Agent 是谁”的问题。到了 Enhanced 阶段，识别只是起点。讲者强调，要把 Agent 当作 first-class citizens，也就是一等主体：身份治理系统不再把 Agent 视为某个项目 API key 背后的匿名调用者，也不只把它塞进一个通用服务账号；系统要让它直接参与授权和治理。

“一等主体”这个词很像编程语言里的“一等函数”：函数如果能被赋值、传参、返回、组合，它就进入了语言的基本表达能力；Agent 如果能被登记、分类、授予、审查、冻结、下线，它才进入了企业身份治理的基本表达能力。只给 Agent 一个技术标识，类似给仓库门口贴了标签；把 Agent 纳入身份治理，才相当于把它写进门禁名册、权限清单、巡检流程和违规处置流程。

一等主体的治理要求

Enhanced 阶段的一等主体要求：Agent 必须能作为独立 principal 参与治理。系统需要知道它是谁、被谁委托、承担什么任务、拥有哪些 entitlement、何时获得凭证、凭证何时失效，以及它的行为是否偏离预期。

这里的身份治理不应被理解成“给 Agent 配一个更高级的账号”。它至少包含四类工程对象。第一是生命周期：Agent 从创建、上线、变更用途到退役，都要有状态。第二是授权权益：Agent 能调用哪些工具、读取哪些数据、启动哪些流程，需要被明确列出。第三是审查：权限是否仍然符合任务职责，是否因为需求漂移而越积越多。第四是执行证据：治理决定必须能回到审计链路中，解释某次访问为什么被允许。

这也解释了为什么本章不能只停留在 NHI。NHI 让 Agent 有名字；一等主体让这个名字进入治理系统。前者解决可识别性，后者解决可管理性。现实系统中，很多安全事故正发生在这两者之间：

日志里能看到某个服务账号调用了数据接口，但没人能说清楚这个 Agent 为什么还拥有该权限、权限由谁批准、是否仍然需要。

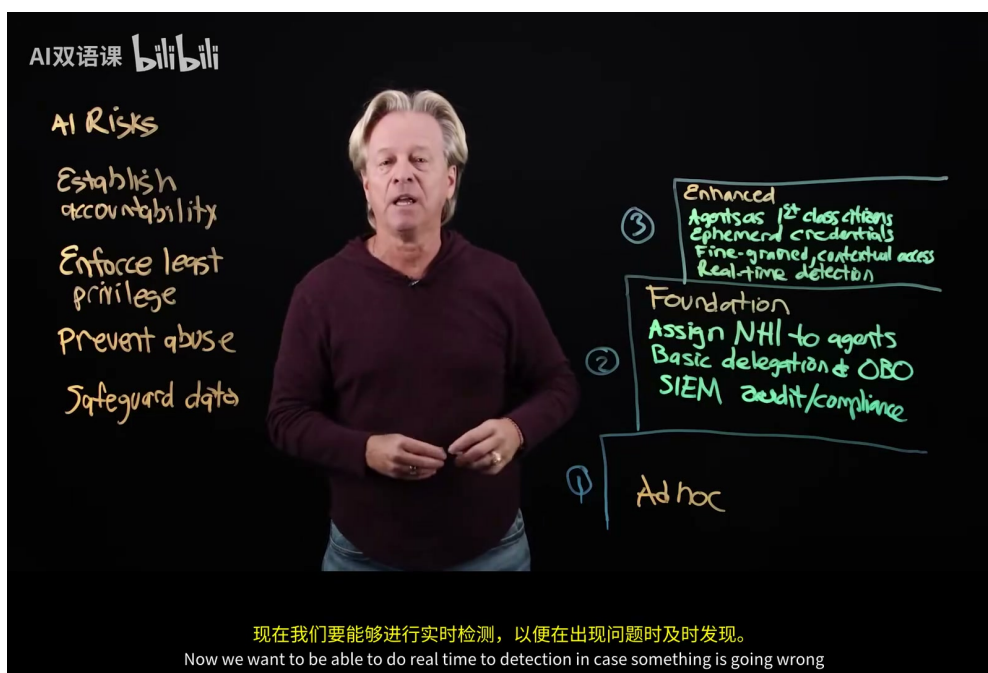


图 7: Enhanced 阶段把 Agent 纳入一等主体治理，并进一步引入临时凭证、精细化上下文访问和实时检测。⁴

类比边界：Agent 与人类身份

一等主体并不表示 Agent 与人类身份完全相同。人类身份通常围绕雇佣关系、岗位、组织架构和人工认证展开；Agent 身份更依赖任务边界、工具链、委托链、部署环境和模型行为。类比关系有教学价值，工程上必须保留差异。

5.2 临时凭证：只为当前任务发放能力

当 Agent 已经成为可治理主体，下一步就是收紧它拿到的凭证。讲者在这一段用 ephemeral credentials 来描述增强阶段的凭证策略：凭证只为 Agent 当前要完成的任务而发放，任务结束后失去效力，范围也不应自然外溢到其他流程。

这与传统应用集成中的长期 API key 很不一样。长期 key 的优点是简单：应用启动后一直能访问目标系统，轮换周期可能是几周或几个月。问题也很直接：一旦 key 被滥用，攻击者继承的是一整段时间、一个较大权限面、甚至多个系统的通行能力。Agentic 系统里，执行路径会变化，长期 key 的风险会被放大，因为系统很难提前穷举 Agent 下一步会碰到哪些资源。

临时凭证把权限从“长期持有”改成“按任务借用”。更具体地说，发放凭证时至少要绑定三个条件：哪个 Agent 使用、为了哪个任务使用、在多长时间窗口内有效。这样做的目标很朴素：即使 Agent 行为异常，或者凭证泄露，凭证本身也只有短寿命和窄用途。

⁴视频画面时间区间：00:10:36–00:10:46。

临时凭证的安全价值

临时凭证的核心价值是降低爆炸半径。它不假设 Agent 永远正确，也不假设凭证永远不会泄露；它通过短生命周期和任务绑定，让一次错误或一次泄露能造成的损害范围变小。

可以把凭证想象成临时施工许可证。永久门禁卡允许持有人长期进出一栋楼；临时施工许可证只允许某个施工队在今天上午进入三楼机房，更换指定设备。Agent 的凭证越接近后者，越符合 Enhanced 阶段的精神。

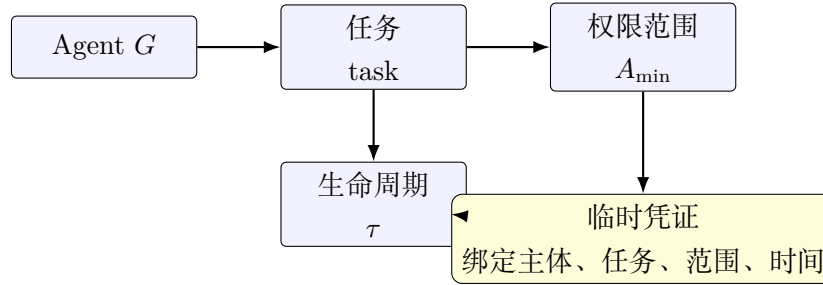


图 8: 临时凭证的四个绑定条件：谁使用、为哪个任务使用、可用权限多大、生命周期多长。

临时凭证还会逼迫系统把“任务”表达清楚。如果企业无法说明某个 Agent 这次调用是为了完成哪一个任务，就很难判断凭证应该包含哪些权限、持续多久、访问哪些资源。这一点经常被低估：短凭证并非单纯把过期时间调短，它要求上游编排系统、IAM 系统和资源系统共同理解任务边界。

凭证轮换与临时凭证的差异

只做周期性凭证轮换不足以达到这里的目标。轮换解决的是“同一把钥匙用太久”的问题；临时凭证进一步要求“这把钥匙只为当前任务存在”。如果权限范围仍然很宽，过期时间从九十天缩到一天也仍然可能过大。

5.3 精细化与上下文访问：按任务、时间与目标收缩权限

Enhanced 阶段随后进入精细化、基于上下文的访问控制。讲者把它放在临时凭证之后，顺序很重要：凭证解决“能力以什么形态交给 Agent”，授权策略解决“能力到底包含什么”。如果凭证短命却权限粗放，Agent 仍然可能在短时间内访问过多系统；如果权限细但凭证长期有效，系统仍然暴露在长期滥用风险中。两者需要一起收紧。

精细化访问控制关注粒度。权限不只写成“可以访问 CRM”或“可以读取数据湖”，还应尽量落到动作、资源、字段、工具、流程步骤等层面。基于上下文的访问控制关注条件。系统要看任务是什么、Agent 当前代表谁、资源敏感度如何、时间窗口是否合理、委托链是否完整、调用环境是否可信。

一个轻量的表达方式如下：

$$A(G, \text{task}, C, R) \subseteq A_{\min}(\text{task}, C, R), \quad \tau(\text{cred}_G) \leq \Delta t_{\text{task}}$$

- G : 正在执行动作的 Agent。
- task : 本次 Agentic flow 要完成的具体任务。
- C : 运行时上下文，包括委托链、时间、环境、资源敏感度和当前流程状态。

- R : 目标资源, 可以是数据、API、工具、服务或业务流程。
- $A(G, \text{task}, C, R)$: 系统实际授予 Agent 的权限集合。
- $A_{\min}(\text{task}, C, R)$: 在该任务、上下文和资源下完成目标所需的最小权限集合。
- $\tau(\text{cred}_G)$: 发给 Agent G 的凭证有效期。
- Δt_{task} : 当前任务合理需要的执行时间窗口。

公式表达的意思很简单: Agent 实际拿到的权限不应超过当前任务所需的最小权限; 凭证寿命不应超过任务合理需要的时间。这里的 $\subseteq$ 是“包含于”的关系, 表示实际权限集合被压在最小权限目标之内。若 $A(G, \text{task}, C, R)$ 比 $A_{\min}$ 大很多, 系统就把额外风险塞给了 Agent; 若 $\tau(\text{cred}_G)$ 远大于 Δt_{task} , 系统就把任务之外的时间也暴露给了凭证。

ABAC 与 Agent 上下文

基于上下文的访问控制与 ABAC (Attribute-Based Access Control, 基于属性的访问控制) 有亲缘关系。ABAC 用主体、资源、动作、环境等属性做授权判断; Agent 场景中的上下文更强调任务状态、委托关系和非确定性路径。可以借用 ABAC 的思想, 但策略输入需要扩展到 Agentic flow。

这类控制在 Agent 场景中尤其关键。传统应用通常有相对固定的调用路径, 长期凭证虽然有风险, 但系统行为较容易预期。Agentic 系统的路径会随目标、工具结果、模型输出和中间状态变化。授权如果只在流程开始时粗略判断一次, 就像只在旅程出发时检查护照, 然后允许旅行者在任何国家、任何设施自由通行。Enhanced 阶段要求每一次关键访问都带着上下文被判断。

举一个具体例子: 一个客服 Agent 代表用户查询订单状态。最低权限可能只需要读取该用户的订单号、物流状态和售后进度。它不应因此获得批量导出所有客户资料的能力, 也不应读取支付令牌、内部风控备注或其他用户订单。若同一个 Agent 后续进入退款流程, 上下文变化了, 授权也要重新计算: 退款金额、账户状态、是否涉及高风险地区、是否超过自动处理阈值, 都会影响允许的动作。

委托不等于全量继承

“Agent 已经代表某个用户”不能自动推出“Agent 继承用户所有权限”。委托关系给了执行理由, 精细化授权决定可执行范围。把两者混在一起, 会让高权限用户无意中制造高权限 Agent。

5.4 实时检测: 从事后日志走向运行中发现异常

Foundation 阶段把日志送入 SIEM, 目的是留下审计和合规证据。Enhanced 阶段继续推进到 real-time detection: 系统开始在 Agentic flow 运行中观察行为, 判断是否出现异常。这里的变化在于时间尺度。事后审计回答“发生了什么”; 实时检测尝试在损害扩大前发现“正在发生什么”。

实时检测并不只寻找恶意攻击。讲者前面已经把滥用分成恶意和无意两类, Enhanced 阶段也要覆盖这两种情形。异常可能是一个被诱导的 Agent 访问了与任务无关的数据, 也可能是开发者设计的流程把 Agent 接到了过宽的系统接口, 还可能是模型在某次运行中选择了一条平时很少出现的工具调用路径。

有效的检测通常需要三类信号。第一是身份信号: 哪个 Agent、代表谁、使用了哪张临时凭证。第二是行为信号: 调用了哪些工具、访问了哪些资源、请求频率是否异常、数据量是否突然放大。第

三是上下文信号：该行为是否符合当前任务、当前时间、当前委托链和资源敏感度。少了身份信号，异常无法归属；少了行为信号，检测没有对象；少了上下文信号，系统会把正常变化误判成风险，或把危险变化当成正常调用。

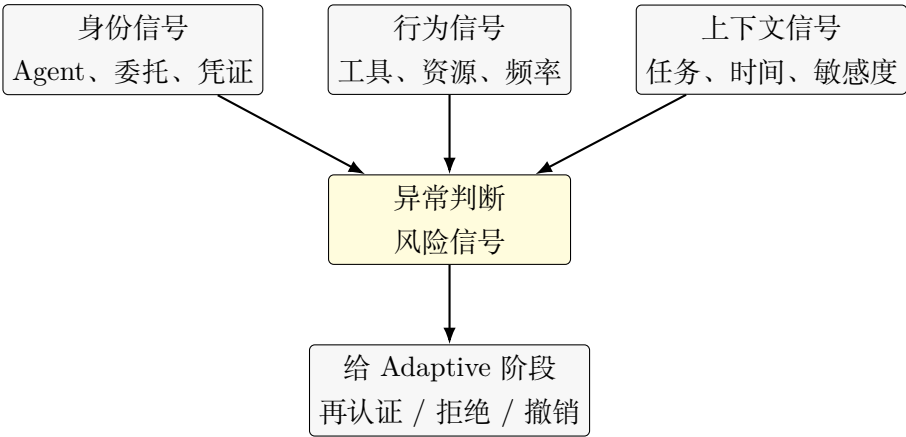


图 9: 实时检测需要把身份、行为和上下文三类信号合在一起，单纯增加日志量不足以形成可执行判断。

实时检测的作用

Enhanced 阶段的实时检测，是在 Foundation 的日志基础上加上运行时判断。日志像行车记录仪，能保存证据；实时检测像主动安全系统，发现偏离车道、异常加速或即将碰撞时立刻发出信号。

不过，实时检测本身仍然只是发现能力。它产生告警、风险信号或异常结论，但还没有天然完成处置。若系统只能在流程结束后生成一条“疑似异常”的记录，Enhanced 的价值会大打折扣。它必须为下一阶段 Adaptive 准备好输入：当检测发现风险升高，后续控制需要能够重新认证、提升验证强度、拒绝下一步访问，或者撤销正在使用的能力。

实时检测超出日志堆积

不要把实时检测误读成“多收集一些日志”。更多日志可能提高可见性，也可能制造噪声。实时检测要求系统把身份、任务、权限、资源和行为连成可判定的信号链，否则告警会变成没人处理的背景噪音。

5.5 本章小结

Enhanced 阶段把 Agentic IAM 从“能看见 Agent”推进到“能治理 Agent”。这一章围绕四个控制点展开：第一，把 Agent 作为一等主体纳入身份治理，让生命周期、权限、审查和执行证据都能围绕 Agent 展开；第二，为 Agent 发放临时凭证，把能力绑定到当前任务和短时间窗口；第三，用精细化、基于上下文的访问控制收缩权限，让授权随任务、资源、时间和委托链变化；第四，在 Foundation 日志之上建立实时检测，尽早发现行为偏离和异常风险。

这四点共同服务于同一个目标：在非确定性的 Agentic flow 中，把权限从“长期、宽泛、静态”压缩为“短期、精确、上下文相关”。但本章也留下了通往 Adaptive 阶段的关键桥梁：检测只有在后续系统能够于流程运行中重新认证、提升验证要求或撤销访问时，才会真正转化为 operational

control。Enhanced 让系统看见异常并量化风险；Adaptive 要把这些信号变成即时执行的身份与访问决策。

6 阶段四：Adaptive，让认证、风险与撤销随流程动态变化

6.1 动态环境：非确定性流程要求持续判断

前一阶段已经把 Agent 当作一等主体来治理：它有自己的身份，使用短生命周期凭证，访问控制会看任务、时间、资源和上下文，还能在运行中做实时检测。到 Adaptive 阶段，问题继续向前推进一步：检测到风险以后，系统能不能马上改变认证强度、访问结果和凭证状态？

这正是演讲者所说的第四步。Agentic 系统的运行环境高度动态，同一个目标在不同执行中可能走出不同路径。传统工作流像预先铺好的铁轨，站点、路线、换乘顺序都比较固定；Agentic 流程更像实时导航，路线会因为工具结果、用户意图、数据敏感度和中途异常不断重算。IAM 在这种系统里也要从“入口检查”升级为“过程控制”。

Adaptive 阶段的核心

Adaptive 阶段把身份、上下文、风险和执行动作连成一个闭环：每一步都重新观察当前状态，必要时提高认证要求，发现异常时撤销访问，并把危险路径阻断在运行中。

这里的“自适应”不等于凭经验放宽权限。它更接近自动驾驶里的安全控制：车辆启动前检查一次驾驶员状态远远不够，系统还要持续感知道路、速度、方向、障碍物和驾驶行为。Agent 也是类似的。它在流程开始时通过认证，只说明起点可信；当它后来访问更敏感的数据、调用更高风险的工具、或者行为偏离预期时，IAM 必须重新判断。

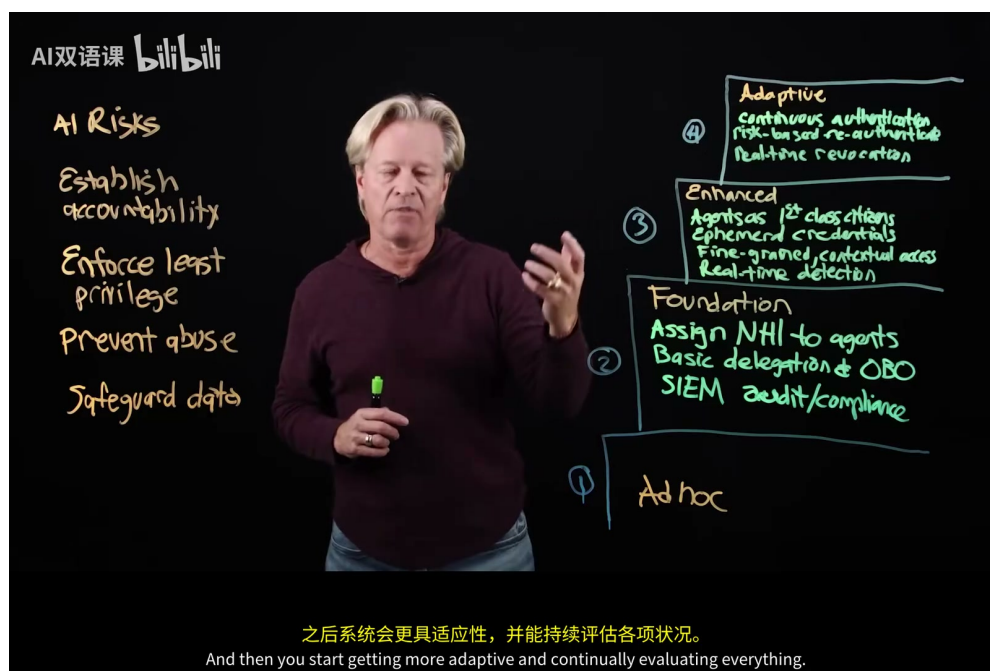


图 10: Adaptive 阶段补上持续认证、基于风险的再认证与实时撤销，形成完整四级成熟度阶梯。⁵

⁵ 视频画面时间区间：00:13:06–00:13:16。

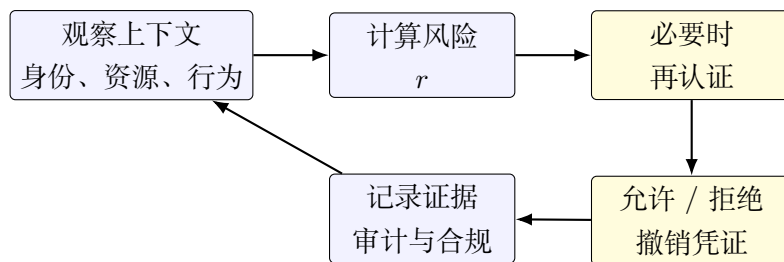


图 11: Adaptive IAM 控制闭环：风险信号必须转化为再认证、拒绝或撤销等运行时动作。

6.2 持续认证：每一步都重新确认 Agent 身份

持续认证指的是在流程进行期间反复确认主体身份，而非只在开始时认证一次。对人类用户来说，这常见于会话风险变化后的重新登录、设备指纹检查、MFA 提示或高风险操作前的二次确认。对 Agent 来说，持续认证要回答更细的问题：此刻调用工具的仍然是这个 Agent 吗？它代表的用户或上游 Agent 仍然有效吗？它手里的凭证还在允许的时间窗内吗？它的任务目标有没有漂移？

把它写成工程语言，就是每个关键步骤都要带上可验证的身份上下文。系统不能只看到“某个服务账号在调用数据库”，还要看到 Agent 身份、代表关系、任务边界、凭证生命周期，以及当前调用发生在哪个上下文中。前面 Foundation 阶段建立的非人类身份和委托链，在这里变成了运行时判断的材料。

持续认证与持续监控的区别

持续监控关注“发生了什么”，例如日志、事件、异常分数和行为轨迹。持续认证关注“当前主体是否仍然可以被确认”，例如身份绑定、凭证有效性、委托关系和会话状态。两者经常配合使用：监控提供风险信号，认证机制决定是否继续信任当前主体。

在 Agentic 流程中，身份常常呈现为一条链。一个用户 U 可以把任务委托给 Agent G_1 ， G_1 又可能调用另一个 Agent G_2 或工具访问资源 R 。持续认证的意义在于：每一次跨边界调用都要保留这条链，避免下游系统只看到一个模糊的自动化账号。

6.3 基于风险的再认证：敏感操作提高验证强度

演讲者接着强调“基于风险的再认证”。它的直觉很简单：所有步骤不应该使用同一种认证强度。读取公开产品文档和导出客户隐私数据，风险完全不同；查询库存状态和发起付款流程，也不能按同一把尺子处理。风险越高，系统越应该要求更强的确认、更短的凭证有效期、更细的授权范围，甚至直接拒绝。

敏感数据风险是 Adaptive 阶段必须显式处理的信号。所谓敏感数据，指泄露或误用后会造成隐私、法律、合规、商业或安全损害的数据。它可以是个人身份信息、客户合同、医疗记录、财务凭证、源代码、密钥材料，也可以是能推导出商业策略的内部分析结果。敏感性越高，访问动作的风险分数 r 越应该上升。

常见误区：认证一次后长期可用

在 Agentic 系统里，把某个 Agent 认证通过后长期放行，会把初始信任误当作全流程信任。演讲者明确指出，访问不应为 Agent 一次性打开后始终可用；每一步都要看目标、上下文和风险。

风险再认证可以理解为一套“台阶式刹车”。低风险步骤继续执行；中等风险步骤要求刷新凭证或确认委托关系；高风险步骤触发更强认证，例如要求上游用户确认、缩短凭证生命周期 τ 、限制可用动作集合 A ，或者把访问结果降级为只读、脱敏、只返回摘要。极高风险步骤则进入拒绝或撤销路径。

这一机制的关键在于避免给每个动作只贴一个静态标签。更可靠的方式是把风险放进上下文 C 里一起计算：谁在执行，代表谁执行，访问什么资源，数据敏感度如何，当前时间是否异常，行为是否偏离历史模式，凭证是否接近过期，委托链是否完整。这样 IAM 才能跟上 Agentic 流程的非确定性。

6.4 实时撤销：检测到异常就切断能力

字幕在 00:11:39 附近出现了“real-time replication”的 ASR 结果，但后文明确说到“real-time revocations”“revoke that access”“put some blocks in place”。因此本讲义采用“实时撤销”。这涉及控制语义的核心：复制访问权会扩大风险，撤销访问权才符合演讲者后续描述。

实时撤销建立在前一阶段的实时检测之上。检测只能告诉系统“这里可能有问题”；撤销让系统真的能“做点什么”。如果 Agent 开始访问与任务无关的资源、试图越权调用工具、重复失败、触发敏感数据策略、或者出现与委托链不一致的行为，IAM 应该能够立即使相关凭证失效、收回会话、停止工具调用，并在流程中插入阻断。

从检测到撤销的控制链

Adaptive 闭环可以压缩为四个动作：检测风险信号，重新评估认证与授权，必要时撤销凭证或访问授权，最后阻断危险路径并留下审计证据。

这里的“凭证”可以是短生命周期 token、访问授权、会话状态、工具调用能力，或者下游系统认可的委托证明。撤销的对象越接近当前任务，影响半径越小。若只能撤销一个共享 API key，很多无关流程会被牵连；若能撤销某个 Agent 在某个任务上的临时凭证，系统就像关闭一扇门，而不会拉掉整栋楼的电闸。

实时撤销还会反向提升最小权限原则。原因很直接：权限越细，撤销越准；凭证越短，残余风险越小；身份传播越完整，系统越知道应该撤销哪一段链路。换句话说，Adaptive 阶段依赖前面几级成熟度打下的地基。

6.5 本章小结

Adaptive 阶段把整个成熟度模型推到运行时闭环。Agentic 系统的非确定性意味着流程中的每一步都可能改变风险形态，因此 IAM 不能停留在入口认证和事后审计上。持续认证让系统反复确认 Agent 身份和委托链；基于风险的再认证让敏感数据和高风险操作获得更强验证；实时撤销让实时检测具备执行后果。

本章最重要的结论是：Adaptive 控制是一条控制链，不应被理解成单独的产品按钮。没有非人类身份，就难以知道该认证谁；没有委托和身份传播，就难以知道 Agent 代表谁；没有短生命周期凭证和精细化访问，就难以精准撤销；没有实时检测，就难以及时触发阻断。

7 闭环：四个成熟度阶段如何回扣四类风险

7.1 从控制回到问责、最小权限、滥用防护与数据保护

演讲者在结尾把四级成熟度路线重新映射回开头的四类风险：问责、最小权限、滥用防护和数据保护。这个回扣很关键，因为成熟度模型的目的不在于堆安全术语，它要让每一类风险都有可落地的控制路径。

第一，问责来自“知道谁在做什么”。Foundation 阶段让 Agent 有非人类身份，Enhanced 阶段让凭证短生命周期化并加强跟踪审计，Adaptive 阶段继续评估每一步。这样系统就能把动作从共享账号、项目级 API key 或模糊服务身份中拆出来，追到具体 Agent、具体委托、具体凭证和具体时间窗。

第二，最小权限来自精细化、基于上下文的访问控制。Agent 只能做它被允许做的事，权限范围应靠近当前任务所需的最小集合 A_{min} 。如果任务只是读取订单状态，就不应拿到修改订单、导出全量客户数据或启动退款流程的能力。这个边界越细，凭证被滥用时的影响半径越小。

第三，滥用防护来自 SIEM、异常检测、错误检测和实时撤销。演讲者特别指出，滥用既包括恶意为行为，也包括无意风险。一个开发者善意地创建了过宽的 Agent 用例，也可能造成破坏。滥用防护因此不能只盯着攻击者，还要覆盖误配置、误授权、提示注入、工具误调用和流程漂移。

第四，数据保护来自身份传播、委托关系和代表执行语义。数据访问判断不能只问“Agent 有没有权限”，还要问“它代表的用户有没有权限”“委托是否仍然有效”“下游系统是否知道这条身份链”。只有把用户、Agent、资源和上下文一起带入判断，敏感数据才不会在自动化链路里被无声绕过。

风险	Foundation	Enhanced	Adaptive
问责	NHI、委托链、SIEM 证据	一等主体治理、短生命周期凭证	持续认证与运行时证据补强
最小权限	识别 Agent 与代表关系	精细化、上下文授权，权限接近 A_{min}	高风险步骤收紧授权或拒绝
滥用防护	日志进入 SIEM，便于追踪	实时检测异常与流程漂移	异常触发再认证和实时撤销
数据保护	同时保留用户与 Agent 语义	按资源敏感度与任务边界授权	敏感访问提高风险阈值

图 12: 四类风险与成熟度控制的对应关系。越往右，控制越接近运行时闭环。

四类风险的闭环映射

问责需要可识别的 Agent、短生命周期凭证、跟踪和审计；最小权限需要精细化上下文授权；滥用防护需要 SIEM、异常检测和实时撤销；数据保护需要身份传播、委托关系和代表执行语义贯穿整个流程。

7.2 渐进路线：先打基础，再持续增强

演讲者最后的提醒很务实：不要试图一次性部署所有控制。成熟度模型的价值就在于把治理任务拆成可推进的阶段。先承认 Ad Hoc 状态，知道当前只是“Agent 能跑起来”；再进入 Foundation，让 Agent 可识别、可委托、可审计；然后进入 Enhanced，把 Agent 作为一等主体治理，缩短凭证生命周期，收紧访问范围，加入实时检测；最后进入 Adaptive，让认证、风险和撤销随流程动态变化。

这条路线对企业尤其重要。很多组织大致知道最终目标，却把最终目标想成一整块大石头，结果无从下手。成熟度模型把它切成台阶：先建立身份，再建立委托，再建立日志，再收紧凭证，再看上下文，再实时检测，再动态撤销。每一步都能减少一部分风险，也都能为下一步提供材料。

凭证控制的回扣

凭证轮换只能降低长期密钥风险；Agentic flow 更依赖短生命周期凭证和实时撤销，因为这两项控制能把能力压到当前任务和当前风险窗口里。

从治理视角看，四级成熟度还对应四种组织能力。Ad Hoc 暴露组织的盲区，Foundation 建立可见性，Enhanced 建立约束力，Adaptive 建立反应力。可见性回答“谁在做”；约束力回答“能做什么”；反应力回答“出问题时能否立即改变结果”。这三种能力缺一项，Agentic IAM 都会留下硬伤。

7.3 本章小结

第 7 节完成了全片的闭环：四个成熟度阶段构成四类风险的治理路线，而非抽象等级。问责依赖身份、凭证和审计；最小权限依赖细粒度上下文授权；滥用防护依赖监控、检测和撤销；数据保护依赖身份传播、委托和代表执行。成熟度越高，系统越能把“事后知道发生了什么”推进到“运行中限制、纠偏和阻断”。

本章同时保留了演讲者的实践态度：从基础开始，逐步增强，不要求组织第一天就拥有 Adaptive 能力。真正危险的是在 Agent 已经能访问工具和数据时，身份、委托、审计、凭证和撤销还停留在 Ad Hoc 状态。

8 总结与延伸

演讲者的收束很清楚：AI Agent 和 Agentic 系统改变了 IAM 的治理对象。传统应用中，团队常用 API key、功能账号或系统账号连接服务；到了 Agentic 场景，系统行为更动态、更非确定，项目级责任已经太粗。治理重心要下沉到 Agent 身份、委托链、凭证生命周期、上下文授权、实时检测和实时撤销。

全片可以浓缩成一个策略函数。自适应 IAM 策略把有效主体、目标资源、运行上下文和风险估计输入到策略中，输出允许、拒绝、再认证或撤销等执行动作：

$$\pi(P, R, C, r) \rightarrow \{\text{allow}, \text{deny}, \text{reauth}, \text{revoke}\}$$

- π ：策略函数，表示 IAM 如何把身份、资源、上下文和风险转换成执行决策。
- P ：有效主体，可以是用户 U 、Agent G ，也可以是委托关系形成的组合主体，例如 $(U \rightarrow G)$ 。
- R ：资源，包括数据、API、工具、服务或业务流程。
- C ：上下文，包括任务、时间、资源敏感度、委托链、环境状态、历史行为和当前执行阶段。
- r ：风险分数，可以是数值等级，也可以是低、中、高等离散风险分类。
- allow：允许访问或继续执行。
- deny：拒绝访问或拒绝执行当前动作。
- reauth：要求重新认证或提高认证强度。
- revoke：撤销凭证、会话、访问授权或工具调用能力。

这个公式的教学价值在于，它把四级成熟度统一起来。Ad Hoc 阶段几乎没有稳定的 π ；Foundation 阶段开始明确 P 和审计证据；Enhanced 阶段收紧 R 、 C 和凭证生命周期；Adaptive 阶段把 r 放进运行时决策，让 reauth 与 revoke 成为流程中的正常动作。

落地时最容易低估的点

Agentic IAM 最容易被低估的是身份链能不能一路传播到下游系统。只要下游丢失了用户、Agent、委托和上下文，授权判断就会退化共享账号判断，审计和撤销也会变粗。

实践上，组织可以按四个问题检查自己的位置。第一，系统能否区分不同 Agent，并把动作追溯到具体身份和委托来源？第二，Agent 的权限是否接近当前任务最小集合，而非拿着长期、宽泛、跨系统的通行证？第三，SIEM、异常检测和实时撤销是否能覆盖恶意与无意滥用？第四，敏感数据访问是否同时校验用户、Agent、资源和上下文？

这些问题还留下几个开放方向。其一，风险分数 r 的计算标准需要组织自己定义，尤其是敏感数据、工具危险度和流程异常的权重。其二，Agent 之间的委托链会越来越长，身份传播格式和审计语义需要足够稳定，否则跨系统追责会断裂。其三，实时撤销必须和业务连续性一起设计，撤销太慢会放大损害，撤销太粗会中断无关流程。其四，成熟度路线要定期复盘，因为 Agent 能力、工具边界和数据资产都会变化。

最终 takeaway

保护 Agent 系统的 IAM 路线可以概括为：先让 Agent 有身份，再让委托可追踪，再让凭证短命、权限精细、检测实时，最后让认证和撤销随着风险动态变化。成熟度模型的意义，是把这条路拆成能执行、能验证、能逐步升级的工程台阶。

落地复盘清单

读者可以把本讲义转成一次内部复盘：列出所有 Agent 和 NHI；标注每条委托链从哪里开始、到哪里结束；检查长期凭证是否可能被任务级短凭证替代；为高敏数据和高风险工具设置再认证条件；确认 SIEM 能看到 Agent、用户、资源、上下文和撤销结果；最后定期回看策略 π 是否仍贴合新的工具、数据和业务流程。

如果这些问题里有任何一项只能靠人工记忆回答，系统仍然保留着 Ad Hoc 的残余风险。成熟度复盘的价值就在这里：它把模糊的安全焦虑拆成一组能被工程团队逐项验证的控制点。